Laboratorio 11. MLOps aplicado al Observatorio de Inteligencia Turística

CC3084 Data Science, Universidad del Valle de Guatemala. Grupo: Esteban Cárcamo, Hugo Barillas, Diego López y Ernesto Ascencio. Entrega final: 1 de noviembre de 2026.

1. Alcance del flujo

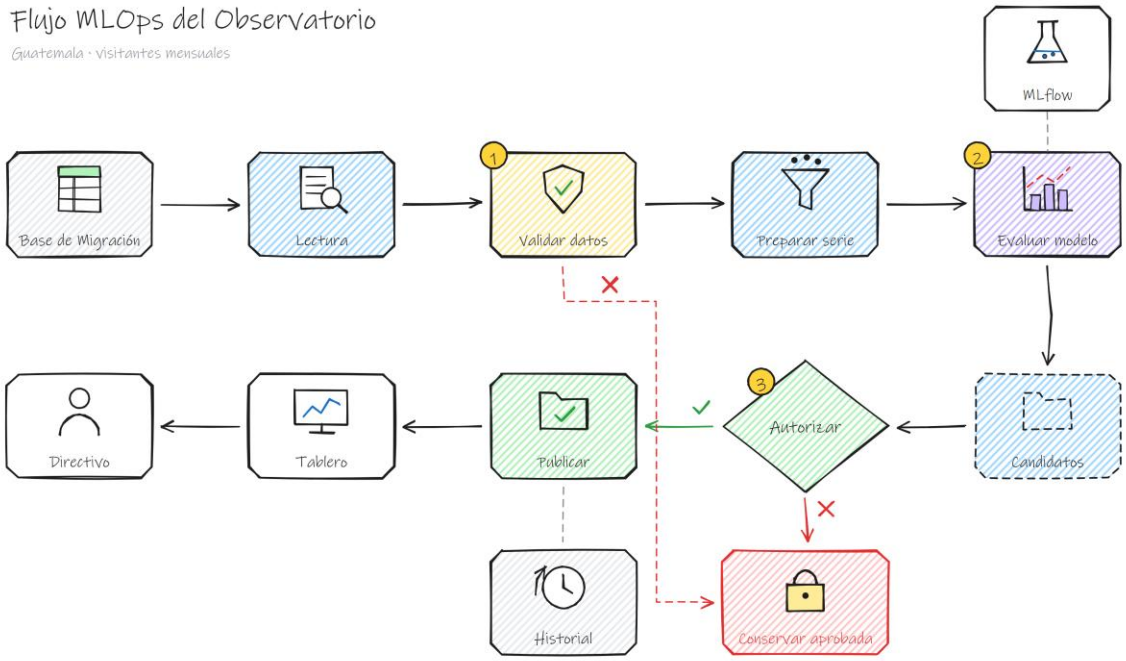
Para este laboratorio decidimos estimar cuántos visitantes internacionales llegan a Guatemala cada mes. Consideramos visitantes a los turistas, que pasan al menos una noche en el país, y a los excursionistas, que lo visitan sin pernoctar. La estimación se expresa en visitantes por mes y abarca todo el país, sin separar por vía de ingreso, frontera ni mercado de origen; por eso, el factor de reparto es igual a 1.

Los datos provienen de la Base de Migración del INGUAT y el IGM, que registra los ingresos al país de enero de 2009 a junio de 2026. De ella utilizamos el año, el mes, la vía, la frontera, el país, la agrupación de residencia y el tipo de viajero, que en conjunto identifican cada registro, además de la cantidad de personas. Dejamos fuera a quienes la base clasifica como «Viajero», porque no cuentan como visitantes, y a los cruceristas, que desde 2023 se miden con otra fuente.

El modelo se ajusta con los datos a partir de enero de 2023, cuando la fuente cambió su forma de contar, y se evalúa entre julio de 2025 y junio de 2026: cada mes se oculta y se estima únicamente con la información anterior. Utilizamos el modelo Holt-Winters del Laboratorio 1 y lo comparamos con una referencia sencilla, que repite el valor del mismo mes del año anterior. Si falta información de algún mes, ese mes nunca se toma como cero: el proceso se detiene.

Las estimaciones que consultará el directivo van de julio a diciembre de 2026. El tablero las lee de un único archivo de resultados aprobados, que solo se reemplaza cuando una nueva versión supera todos los controles de calidad. Todos estos valores quedaron registrados en el archivo de configuración del proyecto, para que cualquier integrante pueda reproducir el flujo.

2. Diagrama del flujo



3. Pipeline reproducible

El pipeline inicial se ejecuta desde la raíz del repositorio con un único comando:

```
python -m pipeline.run --config config/pipeline.yaml
```

El archivo `config/pipeline.yaml` concentra la ubicación y la hoja de la fuente, los tipos de visitantes, la columna de valores, el destino, el factor de reparto, la unidad y las rutas de salida. También conserva los períodos de ajuste y evaluación, los parámetros del procedimiento, la semilla y los umbrales que se utilizarán en las etapas posteriores. Las rutas relativas se resuelven desde la raíz del repositorio, sin depender de archivos intermedios editados manualmente ni del estado de una sesión interactiva.

Etapas implementadas

La ejecución lee la hoja `Datos` del archivo institucional y calcula su huella SHA-256 para identificar la fuente utilizada. A continuación, invoca la función destinada a los controles de calidad y prepara la serie mensual a partir de las filas de turistas y excursionistas, excluyendo viajeros y cruceristas. Los valores se suman por año y mes y se aplica el factor de reparto configurado, cuyo valor para Guatemala es 1.

La serie conserva el histórico disponible y utiliza un índice de inicio de mes. Los meses intermedios sin información permanecen como valores ausentes y no se rellenan con cero. Del

mismo modo, un grupo cuyos valores sean completamente ausentes no se presenta como un total de cero.

Resultados y trazabilidad inicial

Cada ejecución genera un identificador formado por la fecha y hora UTC y un sufijo aleatorio. Los resultados se guardan en `results/candidates/<run_id>/`, sin reemplazar archivos aprobados. El archivo `observed.csv` contiene destino, período, valor, unidad, tipo de dato, identificador de ejecución, fecha de generación y versión del procedimiento configurado; en esta etapa, todos los registros se identifican como observados.

El archivo `run.json` registra las horas de inicio y finalización, las etapas completadas, la huella de la fuente, la ruta de configuración, las advertencias y los errores. Se conserva además una copia del registro en `logs/`. Una falla produce un estado de error y no modifica los resultados aprobados.

4. Controles de calidad 1 a 3

Los controles 1 a 3 están en pipeline/checks.py y son los primeros que ejecuta run_checks. Cada uno devuelve un CheckResult con estado ok, warning o stop; la secuencia se corta en el primer stop y has_stop le indica al pipeline que debe frenar la publicación. Las columnas, la clave y los umbrales salen de las secciones source y quality de config/pipeline.yaml. Los meses sin cobertura no se convierten en cero: se informan como faltantes. Con la base real (161 036 filas) los tres controles dan ok: la serie de visitantes cubre de enero de 2009 a junio de 2026 sin meses faltantes y no hay claves repetidas.

N.º	Control	Qué verifica	Advertencia	Detiene
1	check_columns_and_types (columnas y tipos)	Que estén las columnas indispensables (Año, Mes cod, Vía, Frontera, País, Agrupación Residencia, Tipo de Viajero y Viajero) con tipos compatibles: enteros para Año y Mes cod, texto para las categorías y número para Viajero. También revisa las columnas opcionales (Mes, Región, Región dos,	Falta una columna opcional; el flujo continúa.	Falta una columna indispensable o tiene un tipo incompatible, por ejemplo Viajero como texto.

		Regiones OMT y MCEO).		
2	check_dates_and_coverage (fechas y cobertura)	Que Mes cod esté entre 1 y 12 y que la serie de visitantes (Turista y Excursionista) tenga todos los meses desde enero de 2009 hasta el último período esperado, junio de 2026. Un mes que solo tiene tipos excluidos cuenta como faltante.	Faltan meses antes de enero de 2023, fuera del período de ajuste.	Hay un mes inválido, la fuente no llega a junio de 2026 o falta algún mes desde enero de 2023, dentro del período de ajuste.
3	check_uniqueness (unicidad)	Que no se repita la clave Año, Mes cod, Vía, Frontera, País, Agrupación Residencia y Tipo de Viajero, que define la granularidad de la base. Informa cuántas claves y filas se repiten, con hasta cinco ejemplos.	No aplica con la configuración actual.	Hay al menos una clave repetida.

5. Controles de calidad 4 a 6

Los controles 4 a 6 están en pipeline/checks.py y se ejecutan después de los controles 1 a 3, en el mismo orden, mediante run_checks. El primero que devuelve «stop» corta la secuencia. Cada resultado queda en quality_report.json con el nombre del control, su estado (ok, warning o stop), un mensaje en español y los detalles: conteos por columna, filas inválidas con hasta cinco ejemplos, meses con total cero o meses donde el total recalculado no coincide.

N.º	Control	Qué verifica	Advertencia	Detiene
4	check_missing_values (valores faltantes)	Faltantes por columna, separando las columnas con ausencia permitida (Mes, Región, Región dos, Regiones OMT y MCEO) de las indispensables.	Hay faltantes en una columna permitida.	Hay faltantes en una columna indispensable: Año, Mes cod, Tipo de Viajero, Viajero o cualquiera de la clave única.

5	check_values_and_denominators (valores y denominadores)	Que Viajero sea no negativo y finito, y qué meses de la serie de visitantes suman cero, porque no pueden ser denominador de MAPE ni de la variación interanual.	Hay un mes con total cero: se excluye de MAPE y de la variación interanual, y se informa. Las filas sueltas con cero son normales (hay 54 en la base real) y no generan advertencia.	Hay algún valor negativo, vacío o infinito en Viajero.
6	check_methodological_rule (regla metodológica)	Que la serie salga solo de Turista y Excursionista de la fuente mensual; que ningún tipo excluido (Viajero, Cruceristas) entre al total; que desde enero de 2023 no haya filas de Cruceristas, que se medirían dos veces con la fuente portuaria; y que el total mensual recalculado de forma independiente coincida con la serie preparada.	No aplica.	Se viola cualquiera de esas condiciones.

6. Publicación controlada y ejecución del avance

Si un control crítico devuelve «stop», el pipeline escribe de todos modos el reporte de calidad y el registro de la ejecución (run.json en la carpeta del candidato y una copia en logs/), con estado failed, publicación blocked y la causa: el nombre del control y su mensaje. No prepara la serie, no genera observed.csv, no escribe ni modifica nada en results/approved/ y el comando termina con código de salida 1, de modo que el tablero sigue leyendo la última versión aprobada. Si ningún control detiene el flujo, las advertencias quedan registradas en run.json y la ejecución continúa con la preparación de la serie.

Para la prueba controlada usamos una copia sintética de la base, con la misma estructura y cobertura de enero de 2009 a junio de 2026, y duplicamos una fila para repetir una clave única. Antes de ejecutar colocamos un archivo estimates.csv en la carpeta de resultados aprobados. El pipeline terminó con código 1 y el mensaje «Los controles de calidad detuvieron el flujo. check_uniqueness: Hay 1 claves repetidas.». El reporte marcó check_uniqueness como stop, run.json quedó con estado failed y publicación blocked, no se generó observed.csv y el archivo aprobado conservó la misma huella antes y después de la ejecución.

Luego quitamos la fila duplicada y volvimos a ejecutar: los seis controles dieron ok, el comando terminó con código 0, se generó el candidato con observed.csv y lo aprobado siguió intacto. La misma prueba está automatizada en tests/test_pipeline.py, junto con la de una columna indispensable eliminada.

El avance se ejecuta desde la raíz del repositorio, con la base en data/raw/base_migracion_2009_2026jun.xlsx:

```
pip install -r requirements.txt
```

```
python -m pipeline.run --config config/pipeline.yaml
```

python -m unittest tests.test_checks tests.test_pipeline

7. Registro en MLflow

Cada ejecución del pipeline, exitosa o fallida, se registra en el experimento observatorio-guatemala-visitantes de MLflow Tracking, guardado en la carpeta mlflow/. El run lleva el identificador de la ejecución y registra fechas de inicio y fin, estado, publicación y decisión de aceptación; el commit de Git; la ruta de la fuente y su huella SHA-256; los períodos de ajuste, evaluación y estimación; los parámetros de Holt-Winters y la semilla; MAE, RMSE y MAPE del procedimiento y de la referencia con los meses evaluados; el estado de cada control y de cada verificación de monitoreo; las versiones de Python y de las bibliotecas; y como artefactos los cinco archivos de la ejecución y el YAML de configuración.

Reproducibilidad: ejecutamos dos veces el pipeline completo con la base institucional (huella cdaba779…3846), la misma configuración y las mismas versiones de bibliotecas dentro de Docker: runs 20261102T052257426029Z_c7a4efa9 y 20261102T052540929295Z_bd7b3fe9. El comando python -m pipeline.reproducibility comparó estimaciones, meses evaluados y métricas con tolerancia relativa de 1e-9 y la diferencia máxima fue 0: los resultados son idénticos. El ajuste no tiene componentes aleatorios y la semilla 42 queda fija; la tolerancia solo admitiría redondeos de punto flotante entre equipos distintos, menos de una milésima de visitante.

Ejecuciones fallidas: una ejecución que se detiene antes de evaluar queda en MLflow como FAILED, sin métricas y con la causa en la etiqueta error; no se inventan métricas. La ejecución 20261102T053148655093Z_123e387b, con una copia de la base que tiene una clave duplicada, terminó FAILED con publicación blocked y el error «check_uniqueness: Hay 1 claves repetidas». El listado de ejecuciones, exportado con mlflow.search_runs, está en docs/lab11/mlflow_runs.csv, y la carpeta mlflow/ se entrega comprimida con la versión aprobada. MLflow se consulta en http://localhost:5000.

8. Validación temporal y regla de aceptación

Para validar ocultamos uno a uno los meses de julio de 2025 a junio de 2026 y estimamos cada uno como si no estuviera disponible: Holt-Winters se ajusta solo con los meses desde enero de 2023 hasta el mes anterior al evaluado (horizonte de un mes), de modo que ningún dato del mes evaluado ni posterior interviene. La referencia es el estacional ingenuo, que repite el valor del mismo mes del año anterior, y ambos se evalúan sobre los mismos 12 meses. Usamos MAE y RMSE en visitantes por mes y MAPE; los meses con observado igual a cero se excluirían del MAPE porque el porcentaje no está definido, aunque en la base no hay ninguno.

Resultados con la base institucional: Holt-Winters obtuvo MAE de 16 670, RMSE de 20 068 y MAPE de 6,39 %; la referencia estacional obtuvo MAE de 12 027, RMSE de 15 232 y MAPE de 4,47 %. Holt-Winters no supera a la referencia: acierta mejor en julio, agosto y noviembre de 2025, pero sobreestima nueve de los doce meses, sobre todo en 2026. Al inicio de la evaluación solo hay 30 meses de ajuste para estimar nivel, tendencia y doce índices estacionales, y la tendencia amortiguada todavía arrastra el crecimiento de 2023 y 2024, mientras 2026 se mantuvo cerca de 2025. El mayor error de ambos es abril de 2026 (38 768 visitantes), porque Semana Santa cayó en abril en 2025 y entre marzo y abril en 2026.

Regla de aceptación, fijada antes de ejecutar con la base real: una versión se rechaza si algún control de calidad da stop, si estimates.csv no tiene sus columnas completas o no trae exactamente los seis meses de julio a diciembre de 2026, si hay estimaciones negativas o no finitas, si el MAPE supera 10 % o si el MAE supera 1,5 veces el de la referencia. El 10 % equivale a unos 26 000 visitantes en un mes promedio de 264 790 desde 2023, menos que la variación mensual típica; el 1,5 tolera que con solo 12 meses un mes atípico mueva mucho el MAE, pero rechaza un procedimiento claramente peor que repetir el año anterior.

Aplicación: los seis controles dieron ok, las seis estimaciones están completas y entre 230 737 y 371 750 visitantes, el MAPE es 6,39 % (≤ 10 %) y la razón de MAE es 16 670 / 12 027 = 1,39 (≤ 1,5). La versión se acepta y se publica sin modificar los umbrales. Limitaciones frente a enero y febrero de 2026: la base disponible llega a junio de 2026 y no conservamos la versión que existía a inicios de 2026, así que la evaluación usa valores que pudieron revisarse después; además, con horizonte de un mes febrero se estima con enero ya observado, mientras que en la situación original faltaban ambos meses y el error real sería mayor.

9. Integración con el tablero

El tablero en Dash lee únicamente results/approved/current/, la versión aprobada vigente, y no ajusta ningún modelo: cambiar un filtro solo vuelve a dibujar las gráficas, y los archivos se releen solo cuando cambia run.json tras una publicación o una restauración. Cada fila de estimates.csv incluye destino, período, valor, unidad, tipo (observado o estimado), identificador de ejecución, fecha de generación y versión del procedimiento. G6 muestra lo observado con línea continua y las estimaciones de julio a diciembre de 2026 con línea punteada y etiqueta «Estimado», junto con la fecha de actualización, la versión y el MAPE con una explicación breve y los meses evaluados. G7 compara el error de Holt-Winters con el de la referencia tal como salió.

Debajo de G6 el tablero muestra una advertencia en lenguaje sencillo: son estimaciones puntuales sin margen de error, la incertidumbre crece en los meses más lejanos, no anticipan cambios bruscos como crisis, cierres de frontera o grandes eventos, los meses sin registro se ven como cortes y no se rellenan, y la serie de visitantes usa una definición distinta de la de G1 a G5. Los detalles técnicos de ejecuciones y parámetros quedan en MLflow y en la documentación. La versión aprobada solo se reemplaza después de pasar todas las validaciones, mediante una copia completa a una carpeta temporal y un cambio de nombre atómico; si la publicación falla se conserva la versión anterior, y si aún no existe una versión aprobada el tablero indica que las estimaciones no están disponibles.

10. Monitoreo y recuperación

Cada ejecución escribe run.json en su carpeta candidata y una copia en logs/<run_id>.json, con horas de inicio y fin en UTC, etapas completadas, advertencias, errores, decisión de aceptación y resultado de la publicación (blocked, rejected, published o failed). Después de la regla de aceptación y antes de publicar se ejecutan tres verificaciones de monitoreo contra la versión aprobada vigente. Actualización: la serie debe llegar al último período esperado, junio de 2026, y deben existir los seis meses estimados. Calidad: compara faltantes, filas duplicadas y meses sin cobertura con el reporte de calidad de la versión aprobada y alerta ante cualquier aumento. Resultados: alerta ante estimaciones negativas o no finitas y ante una variación mayor de 30 % frente al mismo mes del año anterior.

El umbral de 30 % sale de la serie de referencia: desde 2025 la variación interanual absoluta tuvo mediana de 4 %, percentil 90 de 13,6 % y máximo de 21,1 %, y el cambio de fecha de Semana Santa mueve marzo y abril entre 13 % y 25 %; 30 % no genera alertas por la estacionalidad habitual. Una variación elevada solo genera una alerta para revisión: no detiene el flujo ni cambia la decisión de aceptación, porque por sí sola no demuestra pérdida de precisión. El error predictivo solo se calcula sobre meses con observación comparable.

Prueba controlada de fallo: con una copia de la base institucional en la que duplicamos una fila (ejecución 20261102T053148655093Z_123e387b) el control de unicidad dio stop, el comando terminó con código 1, run.json quedó con estado failed y publicación blocked, MLflow registró el run como FAILED sin métricas, y la huella de results/approved/current/estimates.csv fue la misma antes y después, de modo que el tablero siguió mostrando la versión aprobada. Corregimos el problema volviendo a la base original y la ejecución 20261102T053256229938Z_0a98251c pasó los seis controles y se publicó.

Alertas demostradas: con una copia en la que dejamos vacía la columna Región en 200 filas (ejecución 20261102T053354747188Z_9d5b791f) el control de faltantes dio advertencia y el monitoreo de calidad alertó «La calidad empeoró frente a la versión aprobada en: valores faltantes»; como Región no interviene en la serie, la versión se aceptó y se publicó con la alerta registrada. Con otra copia en la que triplicamos los valores de junio de 2026 (ejecución 20261102T053453545541Z_9772e746) la regla rechazó la versión porque el MAPE subió a 11,53 %, mayor que el máximo de 10 %, y la versión aprobada no se modificó.

Restauración: cada publicación mueve la versión vigente a results/approved/history/<run_id>/, donde se conservan las versiones anteriores. Para volver a una se ejecuta docker compose run

--rm pipeline python -m pipeline.publish --restore <run_id>. Restauramos la versión 20261102T052257426029Z_c7a4efa9, la primera publicada con la base institucional, y el tablero volvió a consultarla sin reiniciarse: G6 muestra ese identificador de versión y su fecha de actualización.

11. Docker y ejecución

El repositorio incluye un Dockerfile (Python 3.12 y dependencias con versiones fijas en requirements.txt) y un docker-compose.yml con tres servicios: el tablero en el puerto 8050, la interfaz de MLflow en el 5000 y el pipeline, que se ejecuta bajo demanda. La base institucional no entra a la imagen: se coloca en data/raw/base_migracion_2009_2026jun.xlsx, o en otra carpeta indicada con DATA_DIR, y se monta como solo lectura. Las carpetas results/, logs/ y mlflow/ se montan desde el equipo, por lo que los registros de MLflow y las versiones aprobadas persisten al eliminar y volver a crear los contenedores, lo que comprobamos con docker compose down y up.

Pasos desde una copia limpia del repositorio: colocar la base en data/raw/; crear las carpetas con mkdir -p results logs mlflow; construir con GIT_COMMIT=$(git rev-parse HEAD) docker compose build; levantar el tablero y MLflow con docker compose up -d; ejecutar el pipeline con docker compose run pipeline; abrir http://localhost:8050 y http://localhost:5000; restaurar una versión con docker compose run pipeline python -m pipeline.publish --restore <run_id>; y detener con docker compose down. Los registros quedan en logs/, los resultados en results/ y MLflow en mlflow/. El README del repositorio detalla cada paso.

Prueba desde una copia limpia: clonamos la rama lab11 desde GitHub en una carpeta nueva, sin results/, logs/, mlflow/ ni data/, y solo colocamos la base institucional en data/raw/. Con los comandos anteriores la imagen se construyó, el pipeline se ejecutó y publicó una versión aprobada con las mismas seis estimaciones que la copia de trabajo, y el tablero y MLflow respondieron en los puertos 8050 y 5000. Las 140 pruebas automáticas también pasan dentro del contenedor.